



UMCS
UNIwersytet Marii Curie-Skłodowskiej

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **informatyka**

Patryk Wałach

nr albumu: 296597

Porównanie praktyk projektowania CSS: tradycyjne podejście z czystym CSS a nowoczesne narzędzia generujące CSS

Comparative study of CSS design practices:
traditional approach with pure CSS
versus modern CSS generating tools

Praca magisterska

napisana w Katedrze Oprogramowania Systemów Informatycznych

Instytutu Informatyki UMCS

pod kierunkiem **dr hab. Jarosława Byliny**

Lublin 2024

Spis treści

Wstęp	5
1 Ogólne wprowadzenie	7
1.1 Kaskadowe arkusze stylów	7
1.2 Metodologia <i>Blok Element Modyfikator</i>	7
1.3 Semantyczny CSS	8
1.4 Wszechstronny CSS	9
1.5 Statyczny CSS	9
1.6 Programowy CSS	9
2 Najczęściej wykorzystywane funkcjonalności języka CSS	11
2.1 Selektory	12
2.1.1 Selektor elementów	12
2.1.2 Selektor uniwersalny	12
2.1.3 Selektor id	12
2.1.4 Selektor klasy	12
2.1.5 Selektor grupujący	13
2.1.6 Selektor potomków	13
2.1.7 Selektor dziecka	14
2.2 Selektor rodzeństwa	14
2.3 Selektor sąsiedniego rodzeństwa	14
2.4 Pseudo-klasy	14
2.4.1 Funkcyjne pseudo-klasy	15
2.5 Pseudo-elementy	15
2.6 Nadpisywanie stylów	16
2.7 Dziedziczenie stylów	17
2.8 Style warunkowe (ang. <i>media query</i>)	17
2.9 Style dynamiczne	17
3 Problemy języka CSS	19
3.1 Rosnące rozmiary plików	19

3.2	Kolizje pomiędzy stylami	19
3.3	Rozmieszczanie klas pomiędzy plikami CSS	19
4	Narzędzia	21
4.1	Język JavaScript	21
4.1.1	Środowisko Node.js	21
4.2	React	21
4.2.1	Nextjs	21
4.3	Storybook	21
4.3.1	Chromatic	21
5	Funkcjonalności bibliotek TailwindCSS i StyleX	23
5.1	Selektory	23
5.2	Pseudo-klasy	23
5.3	Pseudo-elementy	23
5.4	Nadpisywanie stylów	23
5.5	Dziedziczenie stylów	23
5.6	Style warunkowe (ang. <i>media query</i>)	23
5.7	Style dynamiczne	23
6	Wnioski	25
	Podsumowanie	27
	Bibliografia	29

Wstep

Rozdział 1

Ogólne wprowadzenie

1.1 Kaskadowe arkusze stylów

Kaskadowe arkusze stylów (ang. *Cascading Style Sheets*, w skrócie CSS) jest to deklaratywny język programowania służący do kontrolowania prezentacji dokumentów HTML, XML, SVG itp.

Historia CSS sięga lat 90. XX wieku, kiedy to został stworzony jako odpowiedź na potrzebę oddzielenia struktury dokumentu od jego stylizacji. Wcześniej, wszystkie style były zdefiniowane bezpośrednio w kodzie HTML, co sprawiało, że utrzymanie i zarządzanie stronami internetowymi było trudne i niewydatne.

Za pomocą CSS można kontrolować różne aspekty prezentacji, takie jak kolory, czcionki, marginesy, odstępy między elementami, rozmiar i położenie elementów na stronie, animacje itp.

Dla przykładu, poniższy kod zmienia kolor paragrafu na czerwony:

```
1 p {  
2   color: red;  
3 }
```

Język CSS jest stale rozwijany, co oznacza, że pojawiają się nowe funkcje i możliwości, które pozwalają projektantom jeszcze lepiej kontrolować wygląd swoich stron internetowych. Istnieją również różne wersje CSS, z których najnowsza, CSS3, wprowadza wiele nowych funkcji, takich jak efekty przejścia, animacje, gradienty czy elastyczne rozkładanie elementów.

1.2 Metodologia *Blok Element Modyfikator*

Blok Element Modyfikator (w skrócie BEM) jest to metodologia ułatwiająca tworzenie komponentów wielokrotnego użytku.

Historia BEM sięga 2009 roku, kiedy została wprowadzona przez zespół deweloperów w Mail.Ru, jednej z największych rosyjskich firm internetowych. Jej celem było rozwiązanie problemów związanych z rosnącą złożonością stron internetowych i aplikacji oraz utrzymaniem spójności i czytelności kodu CSS.

Zgodnie z metodologią BEM nazwa każdej z klas CSS musi składać się z trzech elementów:

- Blok (ang. *Block*) - niezależny komponent, który stanowi podstawową część struktury interfejsu. Blok może zawierać inne bloki lub elementy. Każdy blok ma unikalną nazwę, która jest używana w klasach CSS do identyfikacji jego stylów, np. `.header`, `.container`, `.menu`, `.checkbox`, `.input`.
- Element (ang. *Element*) - część składowa bloku, która ma znaczenie tylko w kontekście tego bloku. Elementy są zależne od bloku, w którym się znajdują, i nie mogą istnieć samodzielnie poza nim. Nazwy elementów są prefiksowane nazwą bloku, do którego należą, i oddzielone dwiema podłogami, np. `.menu__item`, `.list__item`, `.checkbox__caption`, `.header__title`.
- Modyfikator (ang. *Modifier*) - odpowiada za zmianę wyglądu lub zachowania bloku lub elementu. Dzięki modyfikatorom możliwe jest dostosowywanie wyglądu i funkcjonalności istniejących bloków i elementów bez konieczności tworzenia nowych. Nazwy modyfikatorów są prefiksowane nazwą bloku lub elementu, do którego się odnoszą, i oddzielone dwoma myślnikami, np. `.btn--disabled`, `.link--highlighted`, `.checkbox--checked`, `.nav--fixed`, `.nav__link--size-big`, `.nav__link--color-yellow`.

1.3 Semantyczny CSS

Semantyczny CSS jest to metodologia, której jednym z największych priorytetów jest "separacja obowiązków" [4].

Idea ta zakłada, że HTML powinien zawierać tylko informacje dotyczące treści, a wszystkie decyzje dotyczące stylizacji powinny być podejmowane w kodzie CSS.

Dla przykładu:

```
1 <p class="text-center">
2   Hello world!
3 </p>
```

Kod zawiera jedną klasę `.text-center`. Centrowanie tekstu to decyzja stylistyczna, więc ten kod łamie zasadę "separacji obowiązków", ponieważ pozwoliliśmy, aby informacje dotyczące stylizacji przeniknęły do naszego HTML-a.

Zamiast tego, zalecane podejście polega na nadawaniu elementom nazw klas na podstawie ich zawartości i wykorzystaniu tych klas jako punktów zaczepienia w kodzie CSS do stylizacji kodu:

```
1 <style>
2 .greeting {
3     text-align: center;
4 }
5 </style>
6
7 <p class="greeting">
8     Hello world!
9 </p>
```

1.4 Wszechstronny CSS

Wszechstronny CSS (ang. *utility-first CSS*) to podejście odwrotne do Semantycznego CSS. Skupia się na tworzeniu małych, pojedynczych klas CSS, z których każda odpowiada za jedną, konkretną właściwość stylu. Zamiast tworzyć specjalne klasy dla konkretnych elementów interfejsu użytkownika, jak nagłówki, przyciski itp. Używa się klas, które definiują pojedyncze właściwości CSS, takie jak kolor, margines czy wypełnienie.

1.5 Statyczny CSS

Statyczny CSS to metodologia w której kod CSS nie zmienia się w trakcie działania aplikacji. Kod CSS jest pisany bez żadnych dodatkowych narzędzi lub generowany na etapie kompilacji po przeanalizowaniu kodu. Pisząc kod w ten sposób, możemy skorzystać z wachlarza narzędzi, analizujących i modyfikujących kod CSS w celu znalezienia błędów, zminimalizowania lub polyfillowania kodu (zastąpienia kodu niewspieranego przez niektóre przeglądarki równoważnym działającym kodem) [1].

1.6 Programowy CSS

Programowy CSS to z kolei metodologia w której style CSS są tworzone w trakcie działania aplikacji. Przewagą tego rozwiązania nad statycznym CSS przez wiele lat była możliwość generowania stylów dynamicznych. Zastosowanie tej metodologii nie wymaga szerokiej znajomości języka CSS, style zwykle definiowane są przy użyciu bibliotek, które zapewniają wysoko poziomowy interfejs. Wykorzystując tego typu biblioteki, w językach

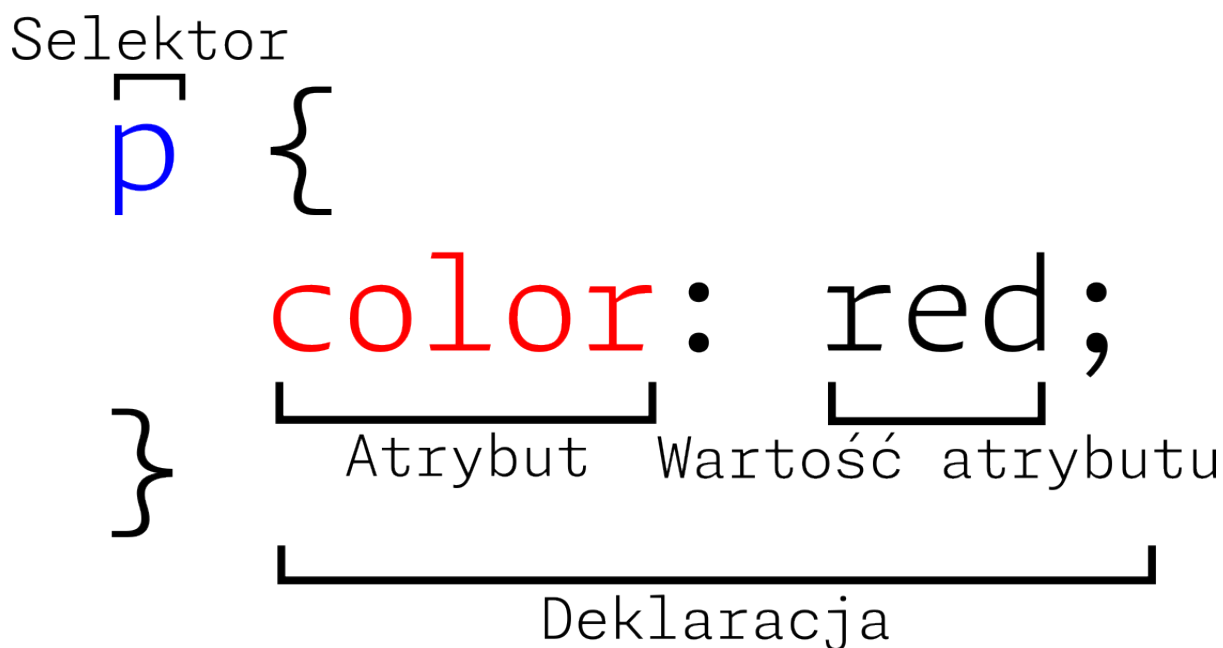
z silnym typowaniem, możemy automatycznie korzystać z autouzupełniania kodu i raportowania błędów w edytorze bez dodatkowych narzędzi, wymaganych do zrozumienia składni języka CSS.

Rozdział 2

Najczęściej wykorzystywane funkcjonalności języka CSS

Podstawowa składnia języka CSS składa się z reguł, których składnie, możemy zaobserwować na rysunku 2.1;

Cała struktura na rysunku 2.1 nazywana jest zestawem reguł. (Termin zestaw reguł jest często nazywany po prostu regułą).



Rysunek 2.1: Przykład reguły CSS, zmieniający kolor paragrafu na czerwony

2.1 Selektory

2.1.1 Selektor elementów

Selektor elementów wybiera elementy HTML na podstawie nazwy elementu. Dla przykładu listing 2.1.

```
1  p {  
2    color: red;  
3  }
```

Listing 2.1: Reguła zostanie zastosowana do wszystkich paragrafów na stronie

2.1.2 Selektor uniwersalny

Selektor uniwersalny wybiera wszystkie elementy HTML na stronie. Dla przykładu listing 2.2.

```
1  * {  
2    color: red;  
3  }
```

Listing 2.2: Reguła zostanie zastosowana do wszystkich elementów na stronie

2.1.3 Selektor id

Selektor id wybiera elementy HTML na podstawie atrybutu id elementu. Id elementu jest unikalne na stronie, dlatego selektor id służy do wyboru jednego unikalnego elementu. Dla przykładu listing 2.3.

```
1  #para1 {  
2    color: red;  
3  }
```

Listing 2.3: Reguła zostanie zastosowana do elementu HTML o atrybucie id="para1"

2.1.4 Selektor klasy

Selektor klasy wybiera elementy HTML na podstawie atrybutu class elementu. Dla przykładu listing 2.4.

```
1  .text-red {  
2    color: red;  
3  }
```

Listing 2.4: Reguła zostanie zastosowana do elementów HTML o atrybucie class="text-red"

Selektor klasy może być używany w połączeniu z selektorem elementu. Dla przykładu listing 2.5.

```
1 p.text-red {  
2   color: red;  
3 }
```

Listing 2.5: Reguła zostanie zastosowana do paragrafów o atrybucie `class="text-red"`

Elementy HTML mogą posiadać wiele klas. Dla przykładu listing 2.6.

```
1 <p class="center large"></p>
```

Listing 2.6: Element będzie stylizowany przez reguły `.center` i `.large`

2.1.5 Selektor grupujący

Selektor grupujący pozwala na połączenie wielu takich samych reguł. Dla przykładu niegrupowane reguły na rysunku 2.7, można zastąpić równoważną regułą tak jak na rysunku 2.8.

```
1 h1 {  
2   color: red;  
3 }  
4 h2 {  
5   color: red;  
6 }  
7 p {  
8   color: red;  
9 }
```

Listing 2.7: Niegrupowane reguły

```
1 h1, h2, p {  
2   color: red;  
3 }
```

Listing 2.8: Pogrupowane reguły

2.1.6 Selektor potomków

Selektor potomków dopasowuje wszystkie elementy będące potomkami określonego elementu. Dla przykładu listing 2.9.

```
1 section p {  
2   color: blue;  
3 }
```

Listing 2.9: Reguła zostanie zastosowana do paragrafów, które są potomkami sekcji

2.1.7 Selektor dziecka

Selektor dziecka wybiera wszystkie elementy będące dziećmi określonego elementu. Dla przykładu listing 2.10.

```
1 section > p {  
2   color: blue;  
3 }
```

Listing 2.10: Reguła zostanie zastosowana do paragrafów, które są dziećmi sekcji

2.2 Selektor rodzeństwa

Selektor rodzeństwa wybiera wszystkie elementy będące rodzeństwem określonego elementu. Dla przykładu listing 2.11.

```
1 p ~ p {  
2   color: blue;  
3 }
```

Listing 2.11: Reguła zostanie zastosowana do paragrafów, umieszczonych za innym paragrafem

2.3 Selektor sąsiedniego rodzeństwa

Selektor sąsiedniego rodzeństwa wybiera element będący sąsiadującym rodzeństwem określonego elementu. Dla przykładu listing 2.12.

```
1 p + p {  
2   color: blue;  
3 }
```

Listing 2.12: Reguła zostanie zastosowana do paragrafów, umieszczonych bezpośrednio za innym paragrafem

2.4 Pseudo-klasy

Pseudo-klasy opisują stany elementów. Na przykład, mogą być użyte do:

- Stylizacji elementu, gdy użytkownik najedzie na niego myszką (listing 2.13)
- Stylizacji odwiedzonych i nieodwiedzonych linków w różny sposób (listing 2.14)
- Stylizacji elementu, gdy otrzymuje skupienie (listing 2.15)

```
1  p:hover {  
2      color: blue;  
3  }
```

Listing 2.13: Reguła zostanie zastosowana do paragrafu, na który użytkownik najedzie myszką

```
1  a:link {  
2      color: red;  
3  }  
4  a:visited {  
5      color: blue;  
6  }
```

Listing 2.14: Reguły zmieniają kolor linku zależnie od jego stanu

```
1  button:focus {  
2      color: red;  
3  }
```

Listing 2.15: Reguła zmienia kolor przycisku, gdy otrzymuje skupienie, np. poprzez wciśnięcie klawisza **Tab**

2.4.1 Funkcyjne pseudo-klasy

Funkcyjne pseudo-klasy to klasy, które akceptują listę selektorów jako parametry. Dla przykładu listing 2.16.

```
1  :not(div):not(.fancy) {  
2      color: blue;  
3  }
```

Listing 2.16: Reguła zostanie zastosowana do elementów, które nie są divami i nie są elementami o atrybucie `class="fancy"`

2.5 Pseudo-elementy

Pseudo-elementy służą do stylizacji określonych części elementów.

Na przykład, mogą być używane do:

- Stylizacji pierwszej litery lub linii elementu (listing 2.17)
- Wstawienia treści przed lub po treści elementu (listing 2.18)

```
1 p::first-letter {  
2   color: red;  
3 }
```

Listing 2.17: Reguła zostanie zastosowana do pierwszej litery każdego paragrafu

```
1 p::before {  
2   content: "Hello";  
3 }
```

Listing 2.18: Reguła doda napis "Hello" do każdego paragrafu

2.6 Nadpisywanie stylów

Jeśli istnieją dwie lub więcej reguł CSS wskazujących na ten sam element, selektor o najwyższej specyficzności "wygra", i jego deklaracja stylu zostanie zastosowana do tego elementu HTML. Dla przykładu listing 2.19.

```
1 <html>  
2 <head>  
3   <style>  
4     .test {color: green;}  
5     p {color: red;}  
6   </style>  
7 </head>  
8 <body>  
9 <p class="test">Hello World!</p>  
10 </body>  
11 </html>
```

Listing 2.19: Tekst paragrafu będzie zielony, ponieważ reguła `.test` ma najwyższą specyficzność

Każdy selektor CSS ma swoje miejsce w hierarchii specyficzności.

Istnieją cztery kategorie, które określają poziom specyficzności selektora:

- Style liniowe - Na przykład: `<h1 style="color: pink;">`
- Identyfikatory - Na przykład: `#navbar`
- Klasy, pseudo-klasy, selektory atrybutów - Na przykład: `.test`, `:hover`, `[href]`
- Elementy i pseudo-elementy - Na przykład: `h1`, `::before`

Tabela 2.1 pokazuje przykłady obliczenia specyficzności.

Tabela 2.1: Przykłady obliczenia specyficzności

Selektor	Specyficzność	Kalkulacja
*	0	0
a	1	1
.btn	10	10
#demo	100	100
<p style="color: pink;">	1000	1000
a.link	11	1 + 10
a.btn.blue	21	1 + 10+10
section#demo	101	1 + 100

2.7 Dziedziczenie stylów

2.8 Style warunkowe (ang. *media query*)

2.9 Style dynamiczne

Rozdział 3

Problemy języka CSS

3.1 Rosnące rozmiary plików

3.2 Kolizje pomiędzy stylami

3.3 Rozmieszczanie klas pomiędzy plikami CSS

Rozdział 4

Narzędzia

4.1 Język JavaScript

4.1.1 Środowisko Node.js

4.2 React

4.2.1 Nextjs

4.3 Storybook

4.3.1 Chromatic

Rozdział 5

Funkcjonalności bibliotek TailwindCSS i StyleX

5.1 Selektory

5.2 Pseudo-klasy

5.3 Pseudo-elementy

5.4 Nadpisywanie stylów

5.5 Dziedziczenie stylów

5.6 Style warunkowe (ang. *media query*)

5.7 Style dynamiczne

Rozdział 6

Wnioski

	An empirical study on the use of CSS preprocessors [3]	Discovering Refactoring Opportunities in Cascading Style Sheets [2]
Zmienne	X	
Zagnieżdżanie	X	
Domieszki	X	
Prefiksy	X	
Deduplikacja		X
Kolejność		X

Tabela 6.1: Kryteria

Podsumowanie

Bibliografia

- [1] B. Lawson and R. Sharp. *Introducing HTML5*. New Riders Publishing, Upper Saddle River, NJ, 2 edition, Oct. 2011.
- [2] D. Mazinianian and N. Tsantalis. Discovering refactoring opportunities in cascading style sheets. <https://people.ece.ubc.ca/amesbah/resources/papers/fse14.pdf>.
- [3] D. Mazinianian and N. Tsantalis. An empirical study on the use of css preprocessors. https://users.encs.concordia.ca/~nikolaos/publications/SANER_2016.pdf, 2016.
- [4] A. Wathan. Css utility classes and "separation of concerns". <https://adamwathan.me/css-utility-classes-and-separation-of-concerns/>, Aug. 2017.